AKTU • B.TECH CSE

DESIGN & ANALYSIS OF ALGORITHMS

UNIT II + UNIT III

MARKS-WISE PYQ SOLUTIONS & REVISION HANDBOOK

Four papers: 2022–23 • 2023–24 • 2024–25 • 2025–26

Exact question wording preserved where legible • concise 2-mark answers • expanded 7/10-mark answers

How to Use This Edition

This is the expanded answer edition. Each PYQ entry includes the paper, section, question number, marks, original wording as printed where legible, and a solution calibrated to the actual marks. The 2022–23 and 2023–24 papers generally use 10-mark long answers; 2024–25 and 2025–26 use 7-mark long answers. The requested "2 short / long detailed" approach is followed, but the actual paper marks are shown rather than relabelled.

The 2025–26 paper is image-based and OCR is imperfect. Any uncertain wording or matrix transcription is flagged. The 2024–25 shortest-path graph is diagram-only; exact weights could not be reliably recovered, so no made-up numeric trace is supplied.

Syllabus map

Unit	Topics
II — Advanced Data Structures	Red-Black Trees; B-Trees; Binomial Heaps; Fibonacci Heaps; Tries; Skip List.
III — Divide and Conquer	Sorting; Matrix Multiplication; Convex Hull; Searching.
III — Greedy Methods	Optimal Reliability Allocation; Knapsack; MST (Prim and Kruskal); single-source shortest paths (Dijkstra and Bellman–Ford).

Classification note: Quick Sort and Heap Sort are listed under Unit I in the supplied syllabus; Quick Sort PYQs are retained in a separate overlap section. Matrix-chain multiplication, Floyd–Warshall and 0/1 Knapsack by dynamic programming are Unit IV, not Unit III.

Complete PYQ locator

Paper	Unit II	Unit III
2022–23	A Q1(c) Skip List (2); A Q1(d) Binomial Tree properties (2); B Q2(b) Binomial vs Fibonacci Heap (10); C Q4(a) RB insertion + height bound (10); C Q4(b) Binomial Heap union (10).	B Q2(a) Merge Sort (10); B Q2(c) MST uniqueness/Kruskal (10); C Q5(a) Fractional Knapsack proof (10); C Q5(b) Dijkstra (10).
2023–24	A Q1(c) Fibonacci Heap (2); B Q2(b) Binomial Tree properties (10); C Q4(a) RB insertion (10); C Q4(b) B-Tree properties/deletion (10).	A Q1(e) Fractional Knapsack (2); A Q1(f) spanning-tree algorithms (2); A Q1(h) Activity Selection (2, related); C Q3(b) Merge Sort (10); C Q6(a) Dijkstra (10).
2024–25	A Q1(d) Binomial Heap properties (2); B Q2(b) Binomial Heap union (7); C Q4(a) Trie/compressed Trie (7); C Q4(b) Binomial Heap construction (7).	A Q1(e) Convex Hull (2); B Q2(a) Merge Sort (7); B Q2(c) shortest path graph (7); C Q5(a) Kruskal (7); C Q5(b) Fractional Knapsack (7).
2025–26	A Q1(f) RB black-height (2); B Q2(e) Fibonacci Heap (7); C Q4(a) Skip List expected level (7).	C Q3(a) MST with vertex 0 as leaf (7); C Q4(b) Kruskal (7); C Q5(b) Dijkstra/negative weights (7). Quick Sort comparison count is Unit I overlap.

PART B — UNIT II: ADVANCED DATA STRUCTURES

1. Red-Black Trees

A Red-Black Tree is a binary search tree with red/black node colours. The invariants keep height logarithmic and therefore search, insertion and deletion are O(log n). Properties: (1) every node red or black; (2) root black; (3) NIL leaves black; (4) red node cannot have red child; (5) every path from a node to descendant NIL leaves has equal black-height.

RB insertion and height proof

PYQ source: 2022–23, Section C Q4(a) | Marks: 10 marks

Exact question as printed: “Discuss the various cases for insertion of key in red-black tree for given sequence of key in an empty red-black tree- {15,13,12,16,19,23,5,8}. Also show that a red-black tree with n internal nodes has height at most 2lg(n+1).”

Exam-ready answer: Insert each key as a BST leaf, initially red. If parent is black, no fix is needed. If parent and uncle are red, recolour both black and grandparent red, then continue upward. If parent is red and uncle black, apply rotations: LL/RR require one rotation; LR/RL require two, with recolouring. Finally force root black. For the bound, a subtree of black-height b has at least 2^b−1 internal nodes, so $n \geq 2^{bh}-1$ and $bh(root) \leq \log_2(n+1)$. Red nodes cannot be consecutive, so at most twice as many total nodes as black nodes occur on a path. Thus $h \leq 2 \cdot bh(root) \leq 2\log_2(n+1)$. For full marks, draw the tree after each insertion and label rotations/recolourings.

RB insertion sequence

PYQ source: 2023–24, Section C Q4(a) | Marks: 10 marks

Exact question as printed: “Insert the following elements using the property of RB tree.
61,58,51,32,39,29”

Exam-ready answer: 61 is black root; 58 is red left child. Insert 51: LL violation, right-rotate at 61 and recolour 58 black, 61 red. Insert 32: parent 51 and uncle 61 are red; recolour 51 and 61 black, 58 red, then restore root black. Insert 39: LR case at 51; left-rotate at 32, then right-rotate at 51, recolouring 39 black and 51 red. Insert 29: parent 32 and uncle 51 are red; recolour 32 and 51 black and 39 red. Show every intermediate tree in the exam and verify all invariants.

Black-height short answer

PYQ source: 2025–26, Section A Q1(f) | Marks: 2 marks

Exact question as printed: “In a Red-Black Tree, how many black nodes are there on any path from the root to the a ans if the black height is 3?”

Exam-ready answer: The black-height is 3, so each root-to-NIL path has black-height 3 under the course convention. State whether the NIL sentinel is counted separately if your instructor distinguishes conventions.

2. B-Trees

A B-Tree is a balanced multiway search tree; keys in each node are sorted and all leaves are at the same depth. With minimum degree t, each node has at most 2t−1 keys and 2t children; each non-root node has at least t−1 keys; an internal root has at least two children; child subtrees cover ranges between separator keys.

B-Tree properties and deletion

PYQ source: 2023–24, Section C Q4(b) | Marks: 10 marks

Exact question as printed: “Explain B-Tree and its properties. Also write B-Tree deletion cases with example.”

Exam-ready answer: Deletion cases: (1) key in leaf: delete if occupancy remains valid. (2) key in internal node: replace by predecessor if left child can spare a key; otherwise successor if right child can spare one; if neither can spare, merge both children with the separator and recurse. (3) key lies below current node: before descent, ensure target child has enough keys by borrowing from a sibling with at least t keys (rotate a separator through parent) or merging with a sibling. If root becomes empty, its only child becomes root. For t=2, a leaf with two keys can lose one; a one-key child must borrow or merge before descent. Draw the affected nodes for full marks.

Brush-up gap: B-Tree insertion did not appear directly. Study splitting a full child, promoting the median key and splitting a full root.

3. Binomial Trees and Binomial Heaps

A binomial tree B_0 is a single node. B_k links two B_{k-1} trees. A binomial heap is a forest of binomial trees satisfying heap order, with at most one tree of each degree.

Binomial Tree properties

PYQ source: 2022–23, Section A Q1(d) | Marks: 2 marks

Exact question as printed: "Discuss the properties of binomial trees."

Exam-ready answer: B_k has 2^k nodes, height k, root degree k, and C(k,i) nodes at depth i. These properties follow recursively from linking two B_{k-1} trees.

Proof of four Binomial Tree properties

PYQ source: 2023–24, Section B Q2(b) | Marks: 10 marks

Exact question as printed: "Prove all the four properties of Binomial Tree."

Exam-ready answer: Induct on k. Base B_0 has one node, height 0, root degree 0 and C(0,0)=1 node at depth 0. B_k links two B_{k-1} trees: node count doubles to 2^k; root gains one child, giving degree k; height increases to k; nodes at depth i equal C(k−1,i)+C(k−1,i−1)=C(k,i) by Pascal's identity. Therefore all four properties hold.

Binomial Heap properties

PYQ source: 2024–25, Section A Q1(d) | Marks: 2 marks

Exact question as printed: "List the properties of Binomial Heap"

Exam-ready answer: Every component is a binomial tree; each tree satisfies min-heap (or max-heap) order; no two trees have the same degree; roots are normally ordered by increasing degree. An n-node heap has O(log n) roots.

Union algorithm and example

PYQ source: 2022–23, Section C Q4(b) | Marks: 10 marks

Exact question as printed: "Explain and write an algorithm for union of two binomial heaps and write its time complexity."

Exam-ready answer: Merge the two root lists in increasing degree. Scan the merged list: if adjacent degrees differ, advance; if three consecutive roots have the same degree, defer linking; otherwise link equal-degree trees, placing the larger root under the smaller for a min-heap. Continue until each degree appears once. Pseudocode: H←merge-root-lists(H1,H2); scan roots x,next; if degree differs or a third root has same degree, advance; else link the larger-key root under the smaller-key root and update the list; return H. Each root list has O(log n) entries, so union is O(log(n+m)), commonly O(log n) for total heap size n.

Binomial Heap definition and union

PYQ source: 2024–25, Section B Q2(b) | Marks: 7 marks

Exact question as printed: "Define Binomial Heap. Write an algorithm for union of two binomial heaps. Also take a suitable example which clearly illustrates merging operation of two binomial heaps."

Exam-ready answer: A Binomial Heap is a forest of heap-ordered binomial trees with unique root degrees. Merge root lists by degree, then link equal-degree roots. For a min-heap, the smaller key becomes parent. Example: degree-0 roots 3 and 7 link into B_1, with 3 as root and 7 as child. If two B_1 trees remain, link them into B_2. Complexity O(log(n+m)); draw the root lists before/after links.

Construct Binomial Heap

PYQ source: 2024–25, Section C Q4(b) | Marks: 7 marks

Exact question as printed: "Design a Binomial Heap for the following A.
A= {7, 2, 4, 17, 1, 11, 6, 8, 15, 10, 20}"

Exam-ready answer: Assume a min-binomial heap and insert keys in the given order, linking equal-degree trees after each insertion. Since $11=8+2+1=2^3+2^1+2^0$, the final forest contains B_3, B_1 and B_0. Ensure every parent key is ≤ its children and no two roots have the same degree. In the exam show each link/insertion state; do not submit only the degree decomposition.

4. Fibonacci Heaps

A Fibonacci Heap is a collection of heap-ordered trees with a circular doubly linked root list and a minimum pointer. It postpones consolidation until extract-min; decrease-key may trigger cuts and cascading cuts.

Operation	Fibonacci Heap	Binomial Heap (typical)
Find-min	O(1)	O(log n) unless pointer maintained
Insert / meld	O(1) amortized	O(log n)
Decrease-key	O(1) amortized	O(log n)
Extract-min	O(log n) amortized	O(log n)

Binomial vs Fibonacci Heap

PYQ source: 2022–23, Section B Q2(b) | Marks: 10 marks

Exact question as printed: "What are the various differences in Binomial and Fibonacci Heap? Explain."

Exam-ready answer: Binomial heaps consolidate during union and maintain at most one tree of each degree. Fibonacci heaps temporarily allow duplicate degrees and postpone consolidation until extract-min. Binomial insert/meld/decrease-key are typically O(log n); Fibonacci insert and meld are O(1) amortized, decrease-key O(1) amortized and extract-min O(log n) amortized. Fibonacci heaps use lazy consolidation and cascading cuts; they are harder to implement but useful in graph algorithms with many decrease-key operations.

Fibonacci Heap short note

PYQ source: 2023–24, Section A Q1(c) | Marks: 2 marks

Exact question as printed: "Write short note on Fibonacci Heap."

Exam-ready answer: A Fibonacci Heap is a priority queue of heap-ordered trees with a minimum pointer. Insert, find-min, meld and decrease-key are O(1) amortized; extract-min is O(log n) amortized. It uses lazy consolidation and cascading cuts.

Fibonacci Heap example

PYQ source: 2025–26, Section B Q2(e) | Marks: 7 marks

Exact question as printed: "Explain Fibonacci Heap with Suitable example."

Exam-ready answer: Insert 7, 3 and 12; they may remain separate roots and the minimum pointer identifies 3. Extract-min removes 3, promotes its children and consolidates roots of equal degree by linking the larger root below the smaller. If decrease-key violates heap order, cut the node into the root list and possibly perform cascading cuts. Amortized costs: insert/meld/decrease-key O(1); extract-min O(log n).

5. Tries and compressed Tries

A Trie stores strings by shared prefixes. A terminal marker denotes a complete word. Search/insert are O(L) for key length L. A compressed Trie merges maximal single-child paths into multi-character edges.

Trie and compressed Trie

PYQ source: 2024–25, Section C Q4(a) | Marks: 7 marks

Exact question as printed: “Insert the following string in the initially empty tries: DOG, DONE, CAT, CAN, RIGHT, DO, JUG, DAA, CA, CAME. Also make a compress tries of it.”

Exam-ready answer: Root branches C, D, J and R. Under C, CA is a complete word; branches lead to CAT, CAN and CAME. Under D, DO is complete; branches lead to DOG and DONE; DAA follows DA then A. JUG and RIGHT each form a single path. In the compressed Trie merge non-branching paths but preserve terminal markers for all ten words. CA and DO must remain terminal even though each has descendants. Draw both structures; mark CAT, CAN, RIGHT, DO, JUG, DAA, CA, CAME, DOG and DONE as complete words.

6. Skip Lists

A Skip List is a probabilistic layered linked list. The base level contains all keys and upper levels contain random subsets. Expected search/insert/delete time is O(log n), worst case O(n).

Skip List and operations

PYQ source: 2022–23, Section A Q1(c) | Marks: 2 marks

Exact question as printed: “Discuss Skip list and its operations.”

Exam-ready answer: Search from the top-left, move right while the next key is smaller, then descend. Insert by finding predecessors at each level and splicing in a node with random height. Delete by unlinking it at each level where it appears. Expected O(log n); worst-case O(n).

Expected nodes at second level

PYQ source: 2025–26, Section C Q4(a) | Marks: 7 marks

Exact question as printed: “In a Skip List, if the Probability of a node being promoted to the next level is 1/2, what is the expected number of nodes at the second level if there are 100 nodes at the base level?”

Exam-ready answer: Let X count promoted nodes. Each of 100 nodes is promoted with probability p=1/2. E[X]=np=100×0.5=50. Expected number at level two is 50; this is an expectation, not a guarantee for a particular random list.

PART C — UNIT III: DIVIDE AND CONQUER

1. Merge Sort

Merge Sort divides the array into halves, recursively sorts each half and merges them. Recurrence $T(n)=2T(n/2)+\Theta(n)$; each level costs $\Theta(n)$ and there are $\log_2 n$ levels, so best/average/worst time is $\Theta(n \log n)$. Typical auxiliary space is $\Theta(n)$; stable if equal keys from the left half are selected first.

```
MERGE-SORT(A,l,r):
  if l >= r: return
  m = floor((l+r)/2)
  MERGE-SORT(A,l,m)
  MERGE-SORT(A,m+1,r)
  MERGE(A,l,m,r)
```

Merge Sort numerical

PYQ source: 2022–23, Section B Q2(a) | Marks: 10 marks

Exact question as printed: "Explain Merge sort algorithm and sort the following sequence {23, 11, 5, 15, 68,31, 4, 17} using merge sort."

Exam-ready answer: Split [23,11,5,15] and [68,31,4,17]. Sort left: [23,11]→[11,23], [5,15]→[5,15], merge→[5,11,15,23]. Sort right: [68,31]→[31,68], [4,17]→[4,17], merge→[4,17,31,68]. Final merge: [4,5,11,15,17,23,31,68]. $T(n)=2T(n/2)+\Theta(n)=\Theta(n \log n)$; auxiliary space $\Theta(n)$. Explain that each recursion level processes n elements and there are $\log_2 n$ levels.

Merge Sort algorithm and proof

PYQ source: 2023–24, Section C Q3(b) | Marks: 10 marks

Exact question as printed: "Write an algorithm of merge sort and prove its worst time complexity."

Exam-ready answer: Use the pseudocode above. $T(n)=2T(n/2)+cn$. At each level, total merge work is cn; the recursion tree has $\log_2 n$ levels. Therefore $T(n)=cn \log_2 n+\Theta(n)=\Theta(n \log n)$. By Master Theorem, a=2,b=2 and f(n)=Θ(n)=n^{log_2 2}, so $T(n)=\Theta(n \log n)$.

Merge Sort array and complexity

PYQ source: 2024–25, Section B Q2(a) | Marks: 7 marks

Exact question as printed: "Illustrate the operation of Merge –Sort on array A= (38, 27, 43, 3, 9, 82, 10). Also drive the time complexity of Merge Sort."

Exam-ready answer: Split into [38,27,43,3] and [9,82,10]. First half sorts to [3,27,38,43]; second to [9,10,82]. Final merge: [3,9,10,27,38,43,82]. Recurrence $T(n)=2T(n/2)+\Theta(n)$; time $\Theta(n \log n)$ in all cases; auxiliary space $\Theta(n)$.

Merge Sort with example

PYQ source: 2025–26, Section B Q2(d) | Marks: 7 marks

Exact question as printed: "Merge Sort with example and ep compute dime complenity."

Exam-ready answer: The scan's English is OCR-imperfect; the legible wording asks for Merge Sort with an example and time complexity. Example: [8,3,6,2] splits into [8,3] and [6,2], becomes [3,8] and [2,6], then merges to [2,3,6,8]. $T(n)=2T(n/2)+\Theta(n)=\Theta(n \log n)$; auxiliary space $\Theta(n)$.

2. Convex Hull

Convex Hull — short answer

PYQ source: 2024–25, Section A Q1(e) | Marks: 2 marks

Exact question as printed: "With a suitable example explain the concept of Convex –Hull Problem"

Exam-ready answer: The convex hull is the smallest convex polygon containing a set of planar points, like a rubber band stretched around the outermost points. Interior points are not hull vertices. Divide-and-conquer approaches find hulls of subsets and merge them using upper/lower tangents.

Brush-up: standard Graham scan/monotone chain runs in O(n log n) after sorting. Divide and conquer splits points by x-coordinate, recursively finds each hull, then merges with common tangents.

3. Matrix multiplication — no direct Unit III PYQ

Standard block divide-and-conquer matrix multiplication uses 8 recursive products plus $\Theta(n^2)$ additions: $T(n)=8T(n/2)+\Theta(n^2)=\Theta(n^3)$. Strassen uses 7 products: $T(n)=7T(n/2)+\Theta(n^2)=\Theta(n^{\log_2 7})\approx\Theta(n^{2.807})$. Matrix-chain multiplication is a different Unit IV dynamic-programming topic.

4. Searching — no direct Unit III PYQ

Binary Search requires sorted input. Compare the middle item with the key, discard half the interval, and repeat. Time O(log n); iterative extra space O(1).

```
BINARY-SEARCH(A,key):
  low=0; high=n-1
  while low<=high:
    mid=floor((low+high)/2)
    if A[mid]==key: return mid
    if A[mid]<key: low=mid+1
    else: high=mid-1
  return NOT_FOUND
```

PART D — UNIT III: GREEDY METHODS

1. Minimum Spanning Tree

An MST of a connected weighted undirected graph connects every vertex without cycles and has V−1 edges with minimum total weight. Kruskal processes edges in ascending order and rejects cycle-forming edges. Prim grows one tree by repeatedly selecting the cheapest crossing edge.

```
KRUSKAL(G):
  sort E by nondecreasing weight
  make-set(v) for every vertex
  T = empty
  for each edge (u,v) in sorted E:
    if find(u) != find(v):
      add (u,v) to T; union(u,v)
    if |T|=|V|-1: break
  return T
```

MST uniqueness and Kruskal

PYQ source: 2022–23, Section B Q2(c) | Marks: 10 marks

Exact question as printed: "Prove that if the weights on the edge of the connected undirected graph are distinct then there is a unique Minimum Spanning Tree. Give an example in this regard. Also discuss Kruskal's Minimum Spanning Tree in detail."

Exam-ready answer: Kruskal sorts edges by weight and adds an edge only when it joins different components; stop at V−1 edges. Complexity O(E log E), dominated by sorting. For uniqueness, suppose distinct weights yet two MSTs T_1, T_2 exist. Let e be the lightest edge in their symmetric difference, $e \in T_1$. Adding e to T_2 creates a cycle containing an edge $f \in T_2 \setminus T_1$ crossing the same cut. Distinct weights and minimality of e imply $w(e)<w(f)$; replacing f by e gives a cheaper spanning tree than T_2, contradiction. Hence the MST is unique. Example: triangle weights 1,2,4 has unique MST using weights 1 and 2.

Spanning tree algorithms — short

PYQ source: 2023–24, Section A Q1(f) | Marks: 2 marks

Exact question as printed: "Write name of Spanning tree algorithm with complexity."

Exam-ready answer: Kruskal: O(E log E), due to sorting edges. Prim: O(E log V) with adjacency lists and binary heap, or $O(V^2)$ with adjacency matrix.

Kruskal with example

PYQ source: 2024–25, Section C Q5(a) | Marks: 7 marks

Exact question as printed: "Write and explain the Kruskal's algorithm to find Minimum Spanning Tree of a graph with a suitable example."

Exam-ready answer: Sort edges ascending; initialise each vertex as a separate component; accept an edge only if it joins different components; union the components; stop at V−1 edges. Example: AB=1, BC=2, AC=3, CD=4. Accept AB, BC; skip AC (cycle); accept CD. MST total = 1+2+4=7. Complexity O(E log E).

Kruskal and time complexity

PYQ source: 2025–26, Section C Q4(b) | Marks: 7 marks

Exact question as printed: "Explain Kruskal's algorithm for MST and also discuss the time complexity."

Exam-ready answer: Sort edges by increasing weight, use disjoint sets to detect cycles, accept edges joining distinct components and stop after V−1 edges. Sorting is O(E log E); union-find costs O(Eα(V)) amortized, so total O(E log E).

MST with vertex 0 as a leaf

PYQ source: 2025–26, Section C Q3(a) | Marks: 7 marks

Exact question as printed: “Consider a complete undirected graph with vertex se 0, 1, 2,3, 4}. W; jin the matrix W below is the weight of the edge {i, j}, What is the minimum possible weight of a spanning tree T in this graph such that vertex O isa leaf node in the tree T?”

Exam-ready answer: The scan matrix is imperfect; the readable symmetric weights appear to be w01=1,w02=8,w03=1,w04=4,w12=12,w13=4,w14=9,w23=7,w24=3,w34=2. Vertex 0 must have degree exactly one; choose a weight-1 edge (0–1 or 0–3). On vertices {1,2,3,4}, choose 3–4=2, 2–4=3 and 1–3=4 for subtotal 9. Add the weight-1 edge from 0: total 10. Verify the matrix entries against the original page image before memorising the numeric answer; method is to remove vertex 0 temporarily, find an MST on remaining vertices, then add the cheapest edge incident to 0.

Prim — syllabus topic not directly asked

Prim begins at a selected root, gives it key 0 and all other vertices infinity, then repeatedly extracts the vertex with minimum key and updates adjacent vertices. Complexity O(E log V) with a heap, or $O(V^2)$ with a matrix.

```
PRIM(G,r):
  key[v]=infinity; parent[v]=NIL
  key[r]=0; Q=vertices keyed by key
  while Q not empty:
    u=extract-min(Q)
    for each edge (u,v,w):
      if v in Q and w<key[v]:
        parent[v]=u; key[v]=w; decrease-key(Q,v,w)
  return parent
```

2. Fractional Knapsack

For each item calculate ratio p_i/w_i, sort in descending order, take whole items while possible and then a fraction of the next. The objective is maximise $\Sigma p_i x_i$ subject to $\Sigma w_i x_i \le M$ and $0 \le x_i \le 1$.

Fractional Knapsack — short

PYQ source: 2023–24, Section A Q1(e) | Marks: 2 marks

Exact question as printed: “Define fractional Knap-Sack problem.”

Exam-ready answer: Given profits p_i, weights w_i and capacity M, choose fractions $x_i \in [0,1]$ to maximise $\Sigma p_i x_i$ subject to $\Sigma w_i x_i \le M$. Items may be divided, unlike 0/1 Knapsack.

Greedy-choice proof and pseudocode

PYQ source: 2022–23, Section C Q5(a) | Marks: 10 marks

Exact question as printed: “Explain “greedy algorithm” Write its pseudo code to prove that fractional Knapsack problem has a greedy-choice property.”

Exam-ready answer: A greedy algorithm chooses the locally best feasible option at each step. Sort by p_i/w_i descending and take as much as possible of each item. Pseudocode: sort items by decreasing p[i]/w[i]; profit=0; for each item, x[i]=min(1,M/w[i]); profit+=x[i]p[i]; M-=x[i]w[i]; stop if M=0. Correctness by exchange argument: if a

solution uses weight from a lower-ratio item while a higher-ratio item is not full, exchange equal weight toward the higher-ratio item. Profit increases or remains unchanged. Repeating yields the greedy solution, so it is optimal. Sorting complexity O(n log n), selection O(n).

Fractional Knapsack numerical

PYQ source: 2024–25, Section C Q5(b) | Marks: 7 marks

Exact question as printed: "Find the optimal solution of the fractional Knapsack problem with n=7 and the knapsack capacity of m=15. The profits and weights of the items are given below.
Objects: 1 2 3 4 5 6 7
Profit (P): 5 10 15 7 8 9 4
Weight (w): 1 3 5 4 1 3 2"

Exam-ready answer: Ratios: item 1=5; 2=10/3≈3.33; 3=3; 4=1.75; 5=8; 6=3; 7=2. Order: 5,1,2,3/6 (tie),7,4. Take 5,1,2,3,6,7 completely. Weight=1+1+3+5+3+2=15; profit=8+5+10+15+9+4=51. Capacity is exactly filled; item 4 is excluded. Optimal profit = 51.

3. Dijkstra — single-source shortest paths

Dijkstra is correct when all edge weights are nonnegative. Initialise d[source]=0 and other distances infinity. Repeatedly extract the unsettled vertex with smallest tentative distance and relax outgoing edges.

```
DIJKSTRA(G,s):
  dist[s]=0; dist[v]=infinity for v!=s; parent[v]=NIL
  Q = priority queue keyed by dist
  while Q not empty:
    u=extract-min(Q)
    for each edge (u,v,w):
      if dist[u]+w < dist[v]:
        dist[v]=dist[u]+w; parent[v]=u; decrease-key(Q,v,dist[v])
  return dist,parent
```

Dijkstra algorithm

PYQ source: 2022–23, Section C Q5(b) | Marks: 10 marks

Exact question as printed: "What are single source shortest paths? Write down Dijkstra's algorithm for it."

Exam-ready answer: Single-source shortest paths find minimum path cost from source s to every reachable vertex. Use the pseudocode above. Relax edge (u,v) by updating d[v] when d[u]+w(u,v)<d[v]. With adjacency list and binary heap, O((V+E)log V); with matrix/array, $O(V^2)$. Dijkstra requires nonnegative edge weights because the minimum tentative distance becomes final when extracted.

Dijkstra worked example

PYQ source: 2023–24, Section C Q6(a) | Marks: 10 marks

Exact question as printed: "Write an algorithm of Dijkstra and implement it by taking any example."

Exam-ready answer: Example edges S→A=4,S→B=1,B→A=2,A→T=1,B→T=5. Initially d(S)=0, others infinity. Process S: A=4,B=1. Process B: A improves to 3, T becomes 6. Process A: T improves to 4. Final distances: S=0,B=1,A=3,T=4. Shortest path to T is S→B→A→T. State the nonnegative-weight condition and complexity O((V+E)log V) using a heap.

Apply greedy shortest path to graph

PYQ source: 2024–25, Section B Q2(c) | Marks: 7 marks

Exact question as printed: "Apply the greedy single source shortest path algorithm on the graph given below."

Exam-ready answer: The question's graph is a printed diagram and its labels could not be reliably reconstructed from the extracted text. Apply Dijkstra to the original graph: initialise source distance 0 and others infinity; repeatedly select the unvisited vertex with the smallest tentative distance; relax every outgoing edge; record predecessors and final distances. Use the diagram on page 1 of the original paper for exact numeric weights. No numeric result is fabricated here.

Dijkstra with negative edges

PYQ source: 2025–26, Section C Q5(b) | Marks: 7 marks

Exact question as printed: "Explain Dijkstra's Algorithm. Prove that it my fail if the graph contains negative edge weights."

Exam-ready answer: Dijkstra assumes the smallest tentative distance becomes final. Counterexample: S→A=2,S→B=5,B→A=−10. Dijkstra can finalise A at 2 before processing B, but the true path S→B→A costs −5. Therefore negative edges can invalidate Dijkstra. Bellman–Ford handles negative edges and detects reachable negative cycles. Heap implementation time is O((V+E)log V), but correctness requires nonnegative weights.

4. Bellman–Ford — syllabus gap

Bellman–Ford allows negative edges. Initialise d[s]=0, others infinity; relax all edges V−1 times; a further successful relaxation indicates a reachable negative cycle.

```
BELLMAN-FORD(G,s):
  dist[s]=0; dist[v]=infinity for v!=s
  repeat |V|-1 times:
    for each edge (u,v,w):
      if dist[u]!=infinity and dist[u]+w<dist[v]:
        dist[v]=dist[u]+w; parent[v]=u
  for each edge (u,v,w):
    if dist[u]!=infinity and dist[u]+w<dist[v]: report negative cycle
  return dist,parent
```

Time O(VE), space O(V). A shortest simple path has at most V−1 edges, so V−1 passes suffice.

5. Optimal Reliability Allocation — syllabus gap

No direct PYQ on this topic was found. The exact formulation depends on the question's reliability model. Typically, allocate a limited budget among components to maximise system reliability. For independent series components, total reliability is the product of component reliabilities. State the objective, constraints, series/parallel structure and reliability function first; then allocate resources based on marginal improvement. Do not apply a memorised formula without matching the given model.

PART E — RELATED OVERLAP: QUICK SORT (UNIT I)

Quick Sort appears in these papers but is listed under Unit I in the supplied syllabus. Included separately because it is a recurring question and a divide-and-conquer algorithm.

Quick Sort best/worst complexity

PYQ source: 2022–23, Section A Q1(b) | Marks: 2 marks

Exact question as printed: "Explain and compare best and worst time complexity of Quick Sort."

Exam-ready answer: Best/average: Θ(n log n) when partitions are balanced. Worst: Θ(n^2) when partitions repeatedly have sizes 0 and n−1, e.g. sorted input with a poor first-element pivot.

Quick Sort partition algorithm

PYQ source: 2024–25, Section C Q3(a) | Marks: 7 marks

Exact question as printed: "Write Quick –Sort partition algorithm. Drive best and worst case time complexity of quick sort."

Exam-ready answer: Partition selects a pivot, moves values ≤ pivot to its left and larger values to its right, then places pivot at final index.

```
PARTITION(A,l,r):
  pivot=A[r]; i=l-1
  for j=l to r-1:
    if A[j]<=pivot: i=i+1; swap A[i],A[j]
  swap A[i+1],A[r]
  return i+1
```

Balanced recurrence T(n)=2T(n/2)+Θ(n)=Θ(n log n). Worst recurrence T(n)=T(n−1)+Θ(n)=Θ(n^2).

Quick Sort comparison counts

PYQ source: 2025–26, Section B Q2(b) | Marks: 7 marks

Exact question as printed: "Let P be a Quick Sort Program to sort numbers in ascending order using the first element as pivot. Let t_1 and t_2 be the number of comparisons made by P for the inputs {1, 2, 3, 4, 5} and {4, 1, 5, 3, 2} respectively. Find the number of t_1 and t_2."

Exam-ready answer: Under the convention that each partition compares pivot to every other element in the current subarray, sorted input gives t_1=4+3+2+1=10. For {4,1,5,3,2}, first partition costs 4 comparisons; recursive partitions cost 2 and 1, so t_2=7. State the implementation convention because exact counts may vary by partition scheme.

PART F — IMPORTANT TOPICS TO BRUSH UP

Priority 1: Repeated / high-yield PYQ topics

- Red-Black insertion, rotations/recolouring, black-height and height theorem.
- Binomial Tree four properties with induction; Binomial Heap union algorithm and complexity.
- Merge Sort traces for both arrays; recurrence and Θ(n log n) proof.
- Kruskal algorithm, cycle avoidance, complexity and uniqueness proof.
- Fractional Knapsack ratios, full numerical and exchange argument.
- Dijkstra trace, complexity and negative-edge counterexample.
- Fibonacci Heap operation complexities and comparison with Binomial Heap.
- Trie/compressed Trie construction with terminal markers; Skip List E[X]=np.

Priority 2: Syllabus topics not directly tested

- B-Tree insertion: split full node, promote median, split root.
- Prim's MST: cut property, algorithm, worked graph, O(E log V) heap / $O(V^2)$ matrix.
- Bellman–Ford: V−1 relaxations, negative edges, negative-cycle detection, O(VE).
- Binary Search: sorted input, pseudocode and O(log n).
- Matrix multiplication: standard $\Theta(n^3)$; Strassen Θ(n^2.807).
- Optimal Reliability Allocation: objective/constraints for the specified series/parallel model.
- Fibonacci Heap extract-min consolidation, decrease-key, cuts and cascading cuts.
- Skip List insertion/deletion, expected O(log n), worst O(n).

How much to write for the marks

2 marks: definition + 2–4 key points or one formula and complexity. Keep it concise. 7 marks: definition, numbered steps/pseudocode, worked example/trace, complexity and short correctness explanation. 10 marks: all of the 7-mark structure plus full proof, every important intermediate tree/table/iteration, and explicit assumptions.

Complexity quick sheet

Topic	Complexity	Exam note
RB Tree operations	O(log n)	$h \le 2\log_2(n+1)$.
B-Tree operations	O(log n) node accesses	Depends on order and I/O model.
Binomial Heap union	O(log n)	O(log n) roots.
Fibonacci insert/meld/decrease-key	O(1) amortized	Extract-min O(log n).
Trie operations	O(L)	L = key length.
Skip List	Expected O(log n), worst O(n)	E[promoted]=np.
Merge Sort	Θ(n log n)	Auxiliary space Θ(n).
Quick Sort	Average Θ(n log n), worst $\Theta(n^2)$	Pivot matters.
Binary Search	O(log n)	Sorted input required.
Kruskal	O(E log E)	Sorting dominates.
Prim	O(E log V) heap; $O(V^2)$ matrix	MST.
Dijkstra	O((V+E)log V) heap	Nonnegative edges only.
Bellman–Ford	O(VE)	Negative edges/cycle detection.
Standard matrix multiply	$\Theta(n^3)$	8 recursive block products.
Strassen	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$	7 recursive products.

Final checklist

- ☐ Draw both RB insertion sequences and explain each fix-up.
- ☐ Prove RB height bound and state black-height convention.
- ☐ Explain B-Tree properties/deletion; revise insertion.
- ☐ Prove binomial-tree properties and write union algorithm.
- ☐ Compare Binomial/Fibonacci heap operations.
- ☐ Draw Trie and compressed Trie for all ten words.
- ☐ Solve Skip List expectation using E=np.
- ☐ Trace both Merge Sort arrays and derive recurrence.
- ☐ Explain Convex Hull, matrix multiplication and Binary Search.
- ☐ Run Kruskal and Prim; solve constrained MST carefully.
- ☐ Solve Fractional Knapsack and prove greedy-choice property.

- ☐ Trace Dijkstra; write Bellman–Ford and negative-cycle test.
- ☐ Review Optimal Reliability Allocation assumptions.

Source integrity notes

Question wording is transcribed from the four supplied papers where legible; line breaks are normalised. The 2025–26 scan contains OCR noise, so uncertain text is flagged. The 2024–25 shortest-path graph cannot be reconstructed reliably from extracted text; use the original page diagram. The 2025–26 MST matrix is also flagged for verification against the scan. The handbook does not claim that past papers guarantee future questions.